

3721. Longest Balanced Subarray II

Solved 

Hard

 Topics

 Companies

 Hint

You are given an integer array `nums`.

A **subarray** is called **balanced** if the number of **distinct even** numbers in the subarray is equal to the number of **distinct odd** numbers.

Return the length of the **longest** balanced subarray.

Example 1:

Input: `nums = [2, 5, 4, 3]`

Output: 4

Explanation:

- The longest balanced subarray is `[2, 5, 4, 3]`.
- It has 2 distinct even numbers `[2, 4]` and 2 distinct odd numbers `[5, 3]`. Thus, the answer is 4.

Example 2:

Input: `nums = [3,2,2,5,4]`

Output: 5

Explanation:

- The longest balanced subarray is `[3, 2, 2, 5, 4]`.
- It has 2 distinct even numbers `[2, 4]` and 2 distinct odd numbers `[3, 5]`. Thus, the answer is 5.

Example 3:

Input: `nums = [1,2,3,2]`

Output: 3

Explanation:

- The longest balanced subarray is `[2, 3, 2]`.
- It has 1 distinct even number `[2]` and 1 distinct odd number `[3]`. Thus, the answer is 3.

Constraints:

- `1 <= nums.length <= 105`
- `1 <= nums[i] <= 105`

Python:

class SegTree:

```
def __init__(self, n):  
    self.n = n  
    self.mn = [0]*(4*n)  
    self.mx = [0]*(4*n)  
    self.lazy = [0]*(4*n)
```

```
def push(self, i):  
    if self.lazy[i]:
```

```

    for c in (2*i, 2*i+1):
        self.mn[c] += self.lazy[i]
        self.mx[c] += self.lazy[i]
        self.lazy[c] += self.lazy[i]
    self.lazy[i] = 0

```

```

def add(self, i, L, R, l, r, v):
    if r < L or R < l:
        return
    if l <= L and R <= r:
        self.mn[i] += v
        self.mx[i] += v
        self.lazy[i] += v
        return
    self.push(i)
    M = (L+R)//2
    self.add(2*i, L, M, l, r, v)
    self.add(2*i+1, M+1, R, l, r, v)
    self.mn[i] = min(self.mn[2*i], self.mn[2*i+1])
    self.mx[i] = max(self.mx[2*i], self.mx[2*i+1])

```

```

def first_zero(self, i, L, R, ql, qr):
    if qr < L or R < ql:
        return -1
    if self.mn[i] > 0 or self.mx[i] < 0:
        return -1
    if L == R:
        return L
    self.push(i)
    M = (L+R)//2
    x = self.first_zero(2*i, L, M, ql, qr)
    if x != -1:
        return x
    return self.first_zero(2*i+1, M+1, R, ql, qr)

```

```

class Solution:
    def longestBalanced(self, nums):
        n = len(nums)
        st = SegTree(n)
        last = {}
        ans = 0

        for r, x in enumerate(nums):

```

```

p = last.get(x, -1)
w = 1 if x % 2 == 0 else -1

st.add(1, 0, n-1, p+1, r, w)

l = st.first_zero(1, 0, n-1, 0, r)
if l != -1:
    ans = max(ans, r-l+1)

last[x] = r

```

```

return ans

```

JavaScript:

```

class SegmentTree {
  constructor(n) {
    this.n = n
    this.minTree = new Array(4 * n).fill(0)
    this.maxTree = new Array(4 * n).fill(0)
    this.lazy = new Array(4 * n).fill(0)
  }

  push(node, start, end) {
    if (this.lazy[node] !== 0) {
      this.minTree[node] += this.lazy[node]
      this.maxTree[node] += this.lazy[node]
      if (start !== end) {
        this.lazy[node * 2] += this.lazy[node]
        this.lazy[node * 2 + 1] += this.lazy[node]
      }
      this.lazy[node] = 0
    }
  }

  updateRange(node, start, end, l, r, val) {
    this.push(node, start, end)
    if (start > end || start > r || end < l) return

    if (l <= start && end <= r) {
      this.lazy[node] += val
      this.push(node, start, end)
      return
    }
  }

```

```

const mid = Math.floor((start + end) / 2)
this.updateRange(node * 2, start, mid, l, r, val)
this.updateRange(node * 2 + 1, mid + 1, end, l, r, val)

this.minTree[node] = Math.min(
  this.minTree[node * 2],
  this.minTree[node * 2 + 1]
)
this.maxTree[node] = Math.max(
  this.maxTree[node * 2],
  this.maxTree[node * 2 + 1]
)
}

findLeftmostZero(node, start, end) {
  this.push(node, start, end)
  if (this.minTree[node] > 0 || this.maxTree[node] < 0) return -1
  if (start === end) return this.minTree[node] === 0 ? start : -1

  const mid = Math.floor((start + end) / 2)
  const left = this.findLeftmostZero(node * 2, start, mid)
  if (left !== -1) return left
  return this.findLeftmostZero(node * 2 + 1, mid + 1, end)
}
}

function longestBalanced(nums) {
  const n = nums.length
  const prev = new Map()
  const st = new SegmentTree(n)
  let res = 0

  for (let r = 0; r < n; r++) {
    const v = nums[r]
    const val = v % 2 === 0 ? 1 : -1

    if (prev.has(v)) {
      st.updateRange(1, 0, n - 1, 0, prev.get(v), -val)
    }

    st.updateRange(1, 0, n - 1, 0, r, val)
    prev.set(v, r)

    const l = st.findLeftmostZero(1, 0, n - 1)

```

```

    if (l != -1 && l <= r) res = Math.max(res, r - l + 1)
}

return res
}

```

Java:

```

class Solution {

    static class Node {
        int l, r;
        int mn, mx;
        int lazy;
    }

    static class SegmentTree {
        Node[] tr;

        SegmentTree(int n) {
            tr = new Node[n << 2];
            for (int i = 0; i < tr.length; i++) {
                tr[i] = new Node();
            }
            build(1, 0, n);
        }

        void build(int u, int l, int r) {
            tr[u].l = l;
            tr[u].r = r;
            tr[u].mn = tr[u].mx = 0;
            tr[u].lazy = 0;
            if (l == r) return;
            int mid = (l + r) >> 1;
            build(u << 1, l, mid);
            build(u << 1 | 1, mid + 1, r);
        }

        void modify(int u, int l, int r, int v) {
            if (tr[u].l >= l && tr[u].r <= r) {
                apply(u, v);
                return;
            }
            pushdown(u);
            int mid = (tr[u].l + tr[u].r) >> 1;

```

```

        if (l <= mid) modify(u << 1, l, r, v);
        if (r > mid) modify(u << 1 | 1, l, r, v);
        pushup(u);
    }

    int query(int u, int target) {
        if (tr[u].l == tr[u].r) {
            return tr[u].l;
        }
        pushdown(u);
        int left = u << 1;
        int right = u << 1 | 1;
        if (tr[left].mn <= target && target <= tr[left].mx) {
            return query(left, target);
        }
        return query(right, target);
    }

    void apply(int u, int v) {
        tr[u].mn += v;
        tr[u].mx += v;
        tr[u].lazy += v;
    }

    void pushup(int u) {
        tr[u].mn = Math.min(tr[u << 1].mn, tr[u << 1 | 1].mn);
        tr[u].mx = Math.max(tr[u << 1].mx, tr[u << 1 | 1].mx);
    }

    void pushdown(int u) {
        if (tr[u].lazy != 0) {
            apply(u << 1, tr[u].lazy);
            apply(u << 1 | 1, tr[u].lazy);
            tr[u].lazy = 0;
        }
    }
}

public int longestBalanced(int[] nums) {
    int n = nums.length;
    SegmentTree st = new SegmentTree(n);
    Map<Integer, Integer> last = new HashMap<>();
    int now = 0;
    int ans = 0;

```

```
for (int i = 1; i <= n; i++) {  
    int x = nums[i - 1];  
    int det = (x & 1) == 1 ? 1 : -1;  
  
    if (last.containsKey(x)) {  
        st.modify(1, last.get(x), n, -det);  
        now -= det;  
    }  
  
    last.put(x, i);  
    st.modify(1, i, n, det);  
    now += det;  
  
    int pos = st.query(1, now);  
    ans = Math.max(ans, i - pos);  
}  
  
return ans;  
}
```